

# How Our Backend Works

## A Beginner's Guide to the Deployment & Workflow

---

No prior DevOps experience needed. This guide explains, in plain language, where our backend code actually runs, how it gets there when we push code, and what happens step-by-step when someone uses the app.

**Think of it this way:** the backend is like a restaurant kitchen. The frontend (Vercel) is the dining room where customers sit. Railway is the kitchen where the food actually gets made. This guide is a tour of that kitchen.

*Skeptic Engine backend — plain-language edition — July 2026*

# What's Inside

1. The Big Picture, In One Paragraph
2. Meet the Pieces (What Each Tool Actually Does)
3. Where Everything Runs: Railway vs. Vercel
4. What Happens When We Push Code
5. What Happens When Someone Uses the App
6. The Settings That Matter Most
7. How We Know If Something's Wrong
8. If Something Breaks: Common Issues
9. Mini Glossary

# 1. The Big Picture, In One Paragraph

Our app has two halves that live in two different places. The part people **see and click on** (the website) is hosted on a platform called **Vercel**. The part that **does the actual work** — storing data, running AI agents, making decisions — is hosted on a platform called **Railway**. They talk to each other over the internet, the same way your phone's weather app talks to a weather server far away.

**Think of it this way:** the website (Vercel) is a TV remote. The backend (Railway) is the TV itself. Pressing a button on the remote doesn't do anything by itself — it sends a signal to the TV, which does the real work and shows you the result.

# 2. Meet the Pieces (What Each Tool Actually Does)

Our backend isn't one single program — it's a small team of tools, each with one job. Here's what each one does, described simply:

| Tool                    | In one sentence                                                                                  |
|-------------------------|--------------------------------------------------------------------------------------------------|
| FastAPI                 | The 'front desk' — receives requests from the website and sends back answers.                    |
| PostgreSQL              | The filing cabinet — stores every case, decision, and record permanently.                        |
| Redis                   | The messenger — passes quick notes between different parts of the backend.                       |
| Celery                  | The back-office team — does the slow, heavy work in the background so the front desk stays fast. |
| EventStoreDB (optional) | The security camera log — keeps an unchangeable record of what happened, for audits.             |
| The AI Agents           | A small panel of 'virtual reviewers' that debate and decide if a transaction looks risky.        |

You don't need to memorize these — just remember that **FastAPI answers requests immediately**, while **Celery does the slow thinking in the background**, and **PostgreSQL never forgets anything**.

### 3. Where Everything Runs: Railway vs. Vercel

Vercel hosts only the website (the part people click on). Railway hosts everything the website depends on. On Railway, our backend is split into **three small copies of the same program**, each one doing a different job:

#### Step A — The API

This is the only one the website talks to directly. It answers questions like 'show me this case' and 'start reviewing this transaction'.

#### Step B — The Worker

This is the 'back office'. When a case needs the AI agents to actually think it through, the Worker is the one that does it — quietly, in the background, without making the website wait.

#### Step C — The Beat

This is the 'alarm clock'. It doesn't answer questions from anyone — it just wakes up on a schedule (like every night) to do routine cleanup jobs.

All three are built from the exact same code — the only difference is the one command each one is told to run when it starts up. That's a deliberate choice: it means the API and the Worker are always running the exact same version of the code, so they never disagree about how things should work.

**Think of it this way:** A, B, and C are three employees who all read the same training manual (the code), but each was given a different job title on their first day.

### 4. What Happens When We Push Code

Getting a code change live is fully automatic — nobody has to log into a server. Here's the journey a change takes:

- **1.** A developer pushes new code to the project's main branch on GitHub.
- **2.** Railway notices the new code right away and starts building a fresh copy of the app for all three services (API, Worker, Beat) at the same time.
- **3.** Before the new API goes live, Railway quietly checks it's healthy by asking it 'are you OK?' at a special web address (/health). Only if it says yes does Railway switch real traffic over to it.
- **4.** If anything's wrong with the new version, the old, working version keeps serving the website — so a bad update doesn't take the app down.
- **5.** Any database changes needed for the new code are applied automatically, right before the API starts.

**Think of it this way:** it's like changing the batteries in a smoke detector while it keeps running on backup power. You don't ever experience a moment where there's no smoke detector at all.

## 5. What Happens When Someone Uses the App

Let's follow one transaction, from the moment someone submits it to the moment a decision comes back.

### Step 1 — Submit

Someone on the website submits a transaction to review. The website sends this to the API on Railway.

### Step 2 — Save & Hand Off

The API saves the basic details straight to the database, then hands the 'go think about this' job to the Worker, and immediately replies to the website: 'got it, working on it'.

### Step 3 — The AI Panel Deliberates

In the background, the Worker runs a small panel of AI agents: one collects evidence, two argue opposite sides (is this risky, or is it fine?), one weighs the argument and decides, and one double-checks that decision is actually backed by evidence.

### Step 4 — Live Updates

While all this happens, the website can watch it happen live (like a progress bar), because the Worker posts updates that the API relays to the browser in real time.

### Step 5 — Final Answer

Once done, the verdict, the full discussion, and the evidence are all saved permanently. If the AI panel isn't confident enough, the case is handed to a real human reviewer instead of being decided automatically.

## 6. The Settings That Matter Most

The backend's behavior is controlled by a list of on/off switches and values called **environment variables**. Think of these as the dashboard settings for the whole app — you don't need to touch the code to change how it behaves, just these values. The handful that matter most day to day:

| Setting         | What flipping it does                                                                                              |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| USE_REAL_AGENTS | Off = practice mode with fake, instant answers. On = the real AI does the thinking (costs money and takes longer). |
| AUTH_REQUIRED   | On = people must log in to use the API. Should always be on in production.                                         |
| DATABASE_URL    | Tells the app which filing cabinet (database) to use.                                                              |
| CORS_ORIGINS    | The list of websites allowed to talk to our API. Must include our real website's address.                          |
| SECRET_KEY      | A private password used to make sure login tokens can't be faked.                                                  |

These are set once, per environment, directly in Railway's dashboard — never written into the code itself, so they can be different for testing vs. the real live app, and so secrets never end up on GitHub.

## 7. How We Know If Something's Wrong

The backend has a few simple ways to 'check its own pulse':

- **/health** — the simplest check. If this responds, the app is at least alive.
- **/health/detailed** — also checks that the database is reachable, and shows how many people are currently connected live.
- **/docs** — a friendly, clickable page listing every single thing the API can do — handy for testing things by hand.
- **Railway's dashboard logs** — a live scrolling feed of everything each service (API / Worker / Beat) is doing or complaining about.

**Think of it this way:** /health is like checking someone has a pulse; /health/detailed is like also checking they can hear and see properly.

## 8. If Something Breaks: Common Issues

| What you notice                                          | What's probably going on                                                                                       |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| The website shows an error saying it can't reach the API | The Railway API service may be down, or its web address changed and the website's settings weren't updated.    |
| Cases stay stuck saying 'in progress' forever            | The Worker service isn't running, or it lost its connection to Redis (the messenger).                          |
| Live progress updates never show up                      | The real-time messenger (Redis) setting is turned off, or the API and Worker aren't pointed at the same Redis. |
| Everyone gets logged out unexpectedly                    | The SECRET_KEY setting changed — this invalidates all existing logins.                                         |
| Scheduled nightly cleanup never happens                  | The Beat service isn't running, or two copies of it are running and fighting each other.                       |

## 9. Mini Glossary

| Term   | Plain-English meaning                                                                |
|--------|--------------------------------------------------------------------------------------|
| Deploy | Publishing a new version of the app so real users get it.                            |
| API    | A way for two programs to talk to each other, like a menu of things you can ask for. |



| Term                 | Plain-English meaning                                                   |
|----------------------|-------------------------------------------------------------------------|
| Backend              | The behind-the-scenes part of an app that does the real work.           |
| Frontend             | The part of an app you actually see and click on.                       |
| Database             | Permanent storage for information, like a digital filing cabinet.       |
| Background job       | Something the app does later, without making you wait for it right now. |
| Environment variable | A setting you can change without editing the actual code.               |
| Health check         | A quick automatic test to see if a service is working.                  |

---

*That's the whole picture: a website on Vercel, a small team of backend services on Railway, and a shared filing cabinet and messenger connecting them all. For deeper technical detail (exact commands, full environment variable list, and file-level references), see the detailed deployment document.*